

Obstacle Avoidance for an Unmanned Ground Vehicle

Omar Mones*, Amr Sayed*,
Zakaria Shamseldin Eleila*, Nooreldin Sameh *, Mohammed Hany * and Asser Mohamed*

**German University in Cairo (GUC), Egypt*

Emails: omar.hendawy@student.guc.edu.eg, amr.behery@student.guc.edu.eg, zakaria.eleila@student.guc.edu.eg, Nooreldin.ahmed@student.guc.edu.eg, mohammed.hany@student.guc.edu.eg, asser.abdallah@student.guc.edu.eg

Abstract—This paper presents the design and implementation of an autonomous unmanned ground vehicle (UGV) capable of path tracking and obstacle avoidance using offline-generated reference trajectories. The system integrates a Raspberry Pi, ESP32, IMU sensor, encoder, and motor control hardware within a ROS2-based architecture. Vehicle localization is achieved using an Extended Kalman Filter (EKF), while velocity and lateral motion are controlled using a P controller and a Stanley controller respectively. The vehicle was tested on three different tracks in both simulation and real-world experiments. Results showed that the EKF improved tracking accuracy and reduced path-following error compared to operation without filtering. The developed platform demonstrates a low-cost and effective solution for autonomous navigation and obstacle avoidance applications.

Keywords—Autonomous; EKF; PID; Stanley .

I. INTRODUCTION

Autonomous vehicles have attracted significant attention over the past two decades, with extensive research devoted to addressing the many challenges facing the autonomous driving industry. These challenges include: Locomotion, Navigation, Localization, Mapping and Control.

This project aims to design and implement a fully autonomous vehicle that is able to track the required path given a set of reference points. A suitable design for the hardware will be chosen, after that algorithms for state estimation and control will be developed. The map is assumed to be given and offline planning will be done to generate the set of points that the vehicle will follow.

This paper is divided as follows: Section II shows previous literature on this topic, section III shows the used hardware system and the circuit diagram, section IV will show the full system overview, section V shows the simulations done on the system and comparing it to real life testing and section VI shows the concluding thoughts on the results of the project.

II. LITERATURE REVIEW

Many papers were done on different modules in the vehicle automation field. For the control module, different meta-heuristic techniques like: Genetic Algorithm(GA), Mesh Adaptive Direct Search (MADS) and Memetics Algorithm (MA) were used in PID tuning for speed control in [1], while a two-layered controller consisting of a Linear Time-Varying

MPC (LTV-MPC) for the upper layer and a Radial Basis Function Neural Network Proportion-Integral-Derivative (RBFNN-PID) for the lower layer was used for lateral control in [2]. For the localization module, different sensor fusion techniques were used like Kalman filter in [3] and particle filters in [4]. In the domain of navigation and planning, recent hybrid approaches have significantly improved obstacle avoidance for constrained vehicle models. For instance, Feng et al. introduces a hierarchical scheme for Automated Guided Vehicles (AGVs) that integrates an optimized Model Predictive Control (MPC) with a hybrid Artificial Potential Field (APF) [5]. This framework establishes three distinct potential fields to represent obstacles, lane lines, and relative velocities, which are then incorporated into the MPC cost function to ensure safe trajectory generation even in dynamic terminal environments. Additionally, Wu et al. explores an end-to-end architecture leveraging attention mechanisms and Long Short-Term Memory (LSTM) models [6]. By applying attention weights to key points along a navigation path, the vehicle can dynamically adjust its route based on real-time multimodal data, such as LiDAR and images, effectively transforming the navigation task into a robust sequential decision-making process. In terms of system architecture, recent research highlights two primary approaches for connecting perception, localization, and control modules. Pfeiffer et al. explores a hybrid architecture that combines traditional modular pipelines with end-to-end learning, aiming to maintain the reliability of separate modules while benefiting from the efficiency of deep learning [7]. This modularity is further detailed by Yurtsever et al., who review the standard functional decomposition where localization and perception feed directly into a centralized control unit to ensure safety through redundancy [8]. Finally, Shao et al. discusses the practical challenges of these architectures, noting that while modular systems are easier to troubleshoot, they often face significant latency compared to integrated end-to-end models [9].

III. HARDWARE DESIGN AND IMPLEMENTATION

In this section, the hardware components, in addition to the circuit diagram are going to be shown.

Table I: Hardware Components Table

Component	Quantity	Price(EGP)	Where to buy
Raspberry PI 5 Model B 8GB	1	6300	Most Electronics
Power Bank	1	2800	Xiaomistore
Micro-Servo Motor	1	90	Makers Electronics
DC Gear Motor With Encoder	1	475	Ampere Electronics
Jumper Wires	20 or more	50	Future Electronics
Vehicle	1	1500	3D-Printed
ESP32	1	412	Amazon
IMU 6050 Sensor	1	650	Ampere Electronics
Breadboard	1	35	Ampere Electronics
L-298 H-bridge	1	70	Ampere Electronics

A. Vehicle Assembly

Figure 1 shows the used car model.

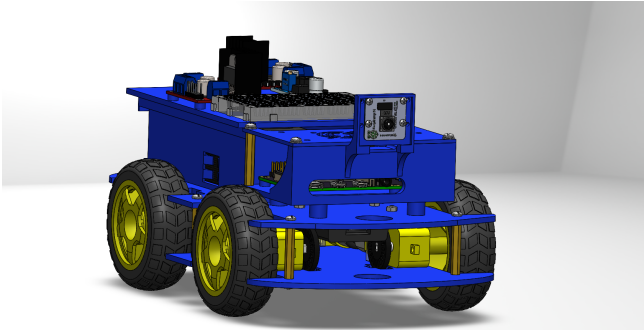


Figure 1: Solidworks Design

B. Hardware Components

Table I shows the components used to build the used vehicle.

C. Circuit Diagram

Figure 2 shows the circuit diagram used in our vehicle.

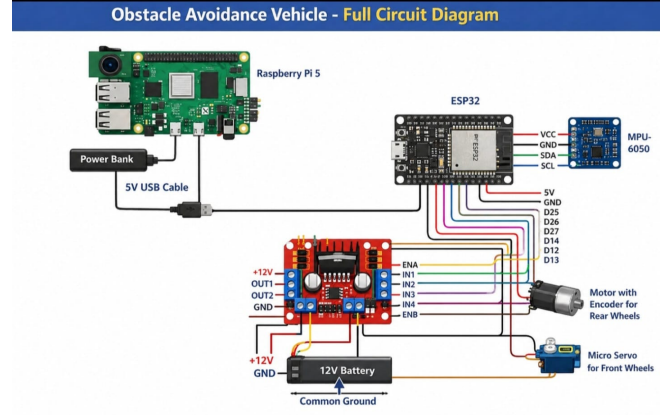


Figure 2: Electric circuit diagram

IV. FULL SYSTEM ALGORITHM

Two micro-controllers are used: Raspberry Pi 5 and ESP32. The ESP32 takes the heading angle from the IMU and the car's speed from the encoder and sends them to the Pi. At the same time, it takes the PWM signal and the steering angle from the Pi and sends it to the DC motor and servo motor respectively.

The Pi consists of 5 nodes which are:

- **Serial Interface node:** This node is responsible for the serial communication between the pi and the ESP32. It takes the sensor data from the ESP32 and sends it to the Localization node, at the same time, it takes the motor commands from the controller nodes and sends them to the ESP32.
- **Reference node:** This node loads the reference points that the vehicle should track from a CSV file, then it publishes the desired X and Y positions, velocities and heading angles to the controller nodes .

- **Localization node:** This node takes the current heading angle and the current velocity from the Serial Interface node and publishes the car's current X-position, Y-position, heading angle and speed. This is done using an Extended Kalman Filter (EKF).
- **Velocity Controller node:** This node subscribes to the outputs of the Reference node and the Localization node. It then calculates the required PWM signal to be given to the DC motor and publishes it to the Lateral Controller node.
- **Lateral Controller node:** This node takes the published outputs from the Velocity Controller node, the Reference node and the Localization node. It then calculates the required steering angle using a stanley controller. Finally, the required steering angle and DC motor's PWM are published to Serial Interface node.

A. Reference Values Generation Algorithm

There are four reference values that are generated: X-position, Y-position, Heading angle and Velocity. The X and Y positions are already given by the loaded waypoints. The reference heading angle is calculated using the following equation:

$$\psi_{ref} = \tan^{-1}\left(\frac{Y_2 - Y_1}{X_2 - X_1}\right) \quad (1)$$

where ψ_{ref} is the desired heading angle, X_n and Y_n are the waypoint X and Y coordinates respectively. The desired velocity is calculated using the following equation:

$$V_{ref} = \frac{V_{max}}{1 + k_c + cur} \quad (2)$$

$$cur = \left| \tan^{-1}\left(\frac{Y_3 - Y_2}{X_3 - X_2}\right) - \tan^{-1}\left(\frac{Y_2 - Y_1}{X_2 - X_1}\right) \right| \quad (3)$$

where V_{ref} is the desired velocity, k_c is the curvature constant and cur is the curvature which uses the next three waypoints to calculate the path's curvature.

B. Localization Algorithm

Localization is done using an Extended Kalman Filter (EKF). An EKF is different from a normal Kalman Filter by introducing the Jacobian matrix F to deal with the model nonlinearities. The used model has three states $[x, y, \theta]$ which are given using the following equations:

$$\begin{bmatrix} x_k \\ y_k \\ \theta_k \end{bmatrix} = \begin{bmatrix} x_{k-1} + v * dt * \cos \theta \\ y_{k-1} + v * dt * \sin \theta \\ \theta_{k-1} + \omega * dt \end{bmatrix} \quad (4)$$

where θ is the heading angle, v is the vehicle's velocity, dt is the time step and ω is the vehicle's angular velocity. The Jacobian matrix is given by the following equation:

$$F = \begin{bmatrix} 1 & 0 & -v * dt * \sin \theta \\ 0 & 1 & v * dt * \cos \theta \\ 0 & 0 & 1 \end{bmatrix} \quad (5)$$

The predicted covariance matrix P_k^- is then calculated using the following equation:

$$P_k^- = F P_{k-1} F^T + Q \quad (6)$$

where P_{k-1} is the covariance matrix calculated in the previous iteration and Q is the model uncertainty.

After that the innovation covariance S is calculated using the following equation:

$$S = H P^- H^T + R \quad (7)$$

where H is the measurement matrix which is $[0, 0, 1]$ since only the yaw is measured directly from the sensors, while R is the sensor uncertainty.

Next the Kalman gain K is calculated using the following equation:

$$K = P^- H^T S^{-1} \quad (8)$$

The Kalman gain is a value between 0 and 1, where the closer the value is to 0 the more the vehicle will trust its model more, while if the value is closer to 1, the vehicle will trust its sensor measurements more.

Finally the states and the covariance matrix are updated using the following equations:

$$X_k = X_k^- + K[Z - H X_k^-] \quad (9)$$

$$P_k = [I - K H] P_k^- \quad (10)$$

where X_k is the actual state vector, X_k^- is the predicted state vector, Z is the measured state vector and I is the identity matrix.

C. Controller Algorithm

Two different controllers are used that control the vehicle's velocity and lateral position which are going to be covered in the following sections:

1) **Velocity Controller:** The velocity is controlled using a P controller given the desired velocity and the current velocity. The required velocity V_{cmd} is calculated using the following equation:

$$V_{cmd} = V_{ref} + V_{corr} \quad (11)$$

where V_{corr} is the output of the P controller.

The required PWM signal is then calculated using the following equation:

$$pwm = pwm_{min} + \frac{V_{cmd}}{V_{max}}(pwm_{max} - pwm_{min}) \quad (12)$$

where pwm_{min} is the minimum pwm signal that moves the vehicle and pwm_{max} is the maximum pwm value (255).

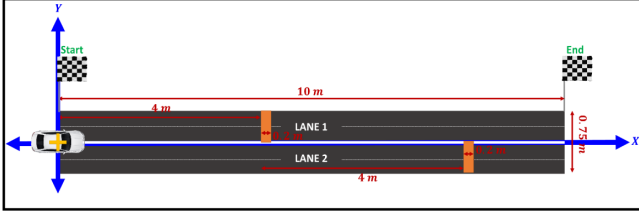


Figure 3: Track 2

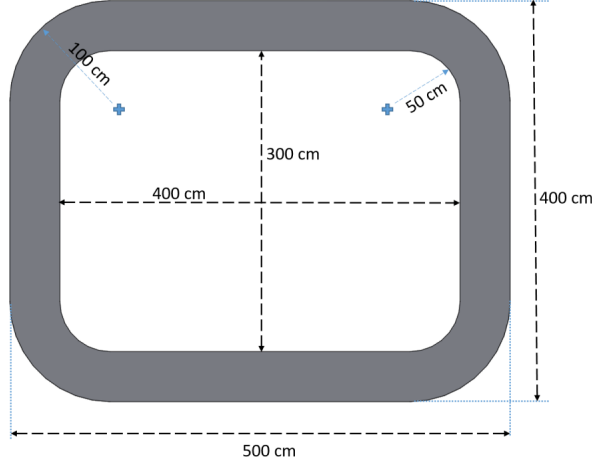


Figure 4: Track 3

2) *Lateral Controller*: The stanley controller is a non-linear controller that uses the car's geometry in order to calculate the required steering angle. It uses the following equation:

$$\theta = \theta_e + \tan^{-1}\left(\frac{k * e_y}{v + \epsilon}\right) \quad (13)$$

$$e_y = -(X_{ref} - X) \sin(\psi) + (Y_{ref} - Y) \cos(\psi) \quad (14)$$

where θ_e is the heading error which is the difference between the desired heading angle (ψ_{ref}) and the current heading angle (ψ), k is the stanley gain, e_y is the cross-track error, v is the current velocity and ϵ is a small value to avoid dividing by zero.

V. SIMULATION RESULTS

Three different maps were used to test the designed vehicle both in simulations and real life. The set of reference points was generated by offline planning using algorithm. The following sections will cover each of these maps separately.

A. Map 1: Straight Line Track

In this map the vehicle will move in a single straight lane of length 10 m. Figure 5 compares the desired path to the actual path covered by the vehicle when not using a Kalman filter. Figure 6 compares the desired path to the actual path covered by the vehicle when using a Kalman filter. The

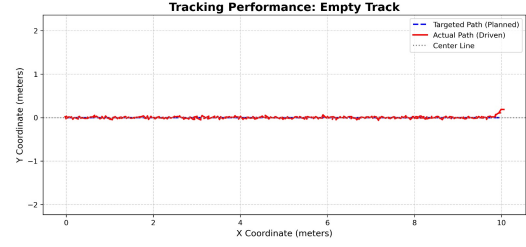


Figure 5: The vehicle following track 1 with no kalman filter

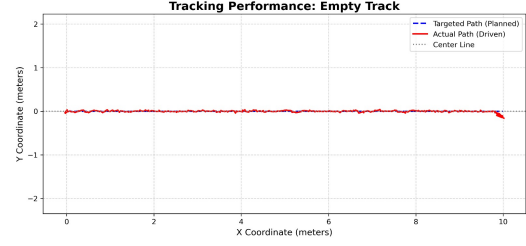


Figure 6: The vehicle following track 1 with a kalman filter

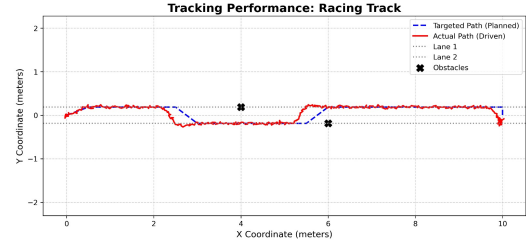


Figure 7: The vehicle following track 2 with no kalman filter

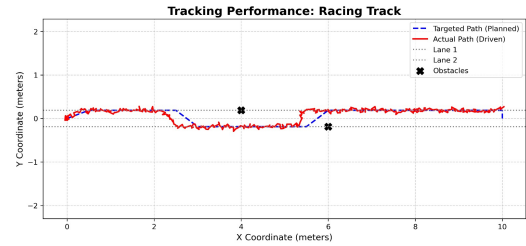


Figure 8: The vehicle following track 2 with a kalman filter

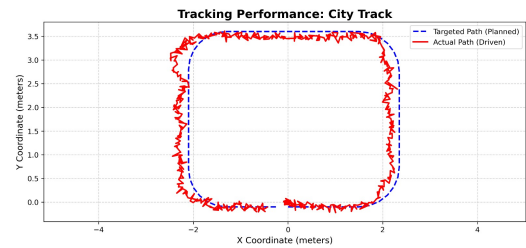


Figure 9: The vehicle following track 3 with no kalman filter

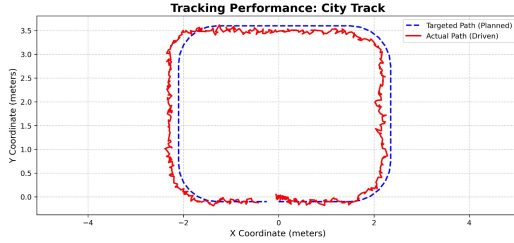


Figure 10: The vehicle following track 3 with a kalman filter

vehicle covered the track with a standard deviation of 2.71 cm in case of no Kalman filter and a standard deviation of 2.31 cm in case of using a Kalman filter.

B. Map 2: Two-Lane Obstacle Avoidance Track

This map consists of two straight lanes. Each lane is 10 m in length with a total width of 0.75 m. Two obstacles are present at distances of 4 and 8 m respectively. Each obstacle is on one of the lanes. The track is shown in figure 3. Figure 7 compares the desired path to the actual path covered by the vehicle when not using a Kalman filter. Figure 8 compares the desired path to the actual path covered by the vehicle when using a Kalman filter. The vehicle covered the track with a standard deviation of 15.11 cm in case of no Kalman filter and a standard deviation of 9.61 cm in case of using a Kalman filter.

C. Map 3: City Track

This map introduces turns that the vehicle should deal with. The track is shown in figure 4. Figure 9 compares the desired path to the actual path covered by the vehicle when not using a Kalman filter. Figure 10 compares the desired path to the actual path covered by the vehicle when using a Kalman filter. The vehicle covered the track with a standard deviation of 146.18 cm in case of no Kalman filter and a standard deviation of 145.83 cm in case of using a Kalman filter.

VI. CONCLUSION

This project presented the design and implementation of an autonomous unmanned ground vehicle capable of path tracking and obstacle avoidance using a modular ROS2-based architecture. The developed system integrated hardware and software components including sensor interfacing, localization, reference generation, and control. An Extended Kalman Filter (EKF) was used to improve vehicle localization, while a P controller and Stanley controller were implemented for velocity and lateral control respectively. Experimental testing on different tracks demonstrated the vehicle's ability to autonomously follow predefined trajectories and avoid obstacles successfully. The obtained results showed that the EKF improved tracking performance and reduced localization error, particularly

in simpler track configurations. Although larger errors appeared in more complex tracks containing sharp turns, the overall system achieved reliable autonomous navigation using low-cost hardware components. Future work may focus on improving controller performance, integrating additional perception sensors such as LiDAR or cameras, and implementing more advanced planning and control techniques to enhance robustness in complex environments.

REFERENCES

- [1] J. E. Naranjo, F. Serradilla, and F. Nashashibi, "Speed control optimization for autonomous vehicles with metaheuristics," *Electronics*, vol. 9, no. 4, p. 551, 2020.
- [2] Z. He, L. Nie, Z. Yin, and S. Huang, "A two-layer controller for lateral path tracking control of autonomous vehicles," *Sensors*, vol. 20, no. 13, p. 3689, 2020.
- [3] Y. Wang, D. Liu, and E. Matson, "Accurate perception for autonomous driving: Application of kalman filter for sensor fusion," in *2020 IEEE sensors applications symposium (SAS)*. IEEE, 2020, pp. 1–6.
- [4] M. Ghiliss, L. Kechiche, and K. Chabir, "Real-time state estimation and security in autonomous vehicles using particle filters," in *2024 IEEE International Multi-Conference on Smart Systems & Green Process (IMC-SSGP)*. IEEE, 2024, pp. 1–6.
- [5] J. Feng, Y. Yang, H. Zhang, S. Sun, and B. Xu, "Path planning and trajectory tracking for autonomous obstacle avoidance in automated guided vehicles at automated terminals," *Axioms*, vol. 13, no. 1, p. 27, 2024.
- [6] X. Wu, G. Wang, and N. Shen, "Research on obstacle avoidance optimization and path planning of autonomous vehicles based on attention mechanism combined with multimodal information decision-making thoughts of robots," *Frontiers in Neurorobotics*, vol. 17, p. 1269447, 2023.
- [7] M. Pfeiffer *et al.*, "Integrating modular pipelines with end-to-end learning: A hybrid approach for robust and reliable autonomous driving systems," *MDPI Sensors*, 2024.
- [8] E. Yurtsever *et al.*, "Autonomous driving: A review of localization, perception, and control," *IEEE Access*, vol. 8, pp. 115 735–115 773, 2020.
- [9] H. Shao *et al.*, "Practical autonomous driving: A survey of challenges and opportunities," *arXiv preprint*, 2023.